Exercice 3 – Modélisation UML

Gestion d'un personnel universitaire

Diagramme de classes · Diagramme d'héritage · Diagramme d'objets · Diagramme de séquence

1) Diagramme de classes

La classe **Employe** (abstraite) regroupe ce qui est commun à tout le personnel : matricule, nom, prénom et salaire de base. Les trois classes dérivées ajoutent les indemnités ou primes propres à leur catégorie. Les flèches à pointe triangulaire creuse représentent l'héritage.

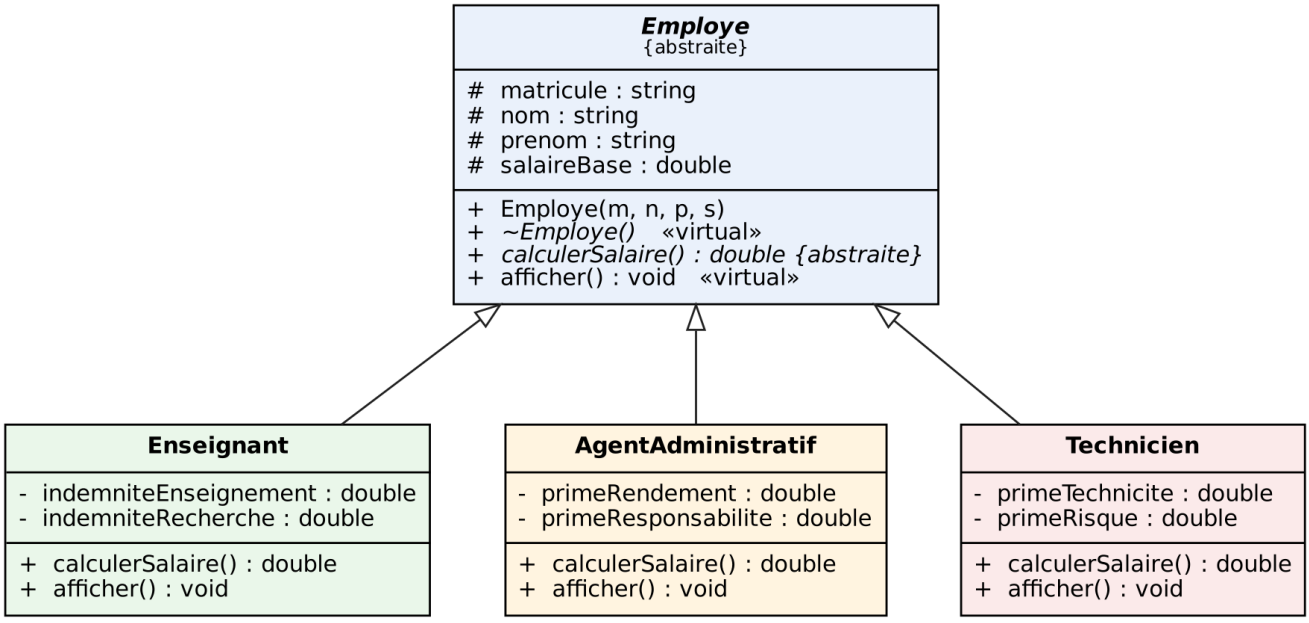


Figure 1 – Diagramme de classes (# protégé, - privé, + public ; italique = abstrait)

Lecture : `calculerSalaire()` est une méthode virtuelle pure dans *Employe* (d'où la classe abstraite) ; chaque classe dérivée la redéfinit. `afficher()` est virtuelle et également redéfinie. Le destructeur de *Employe* est virtuel.

2) Diagramme d'héritage (relations d'héritage)

Ce diagramme montre la **hiérarchie** : la classe mère en haut, les trois classes dérivées en dessous, reliées par une **flèche à pointe triangulaire creuse** orientée de la classe dérivée vers la classe mère. Chaque relation se lit « *est un* ».

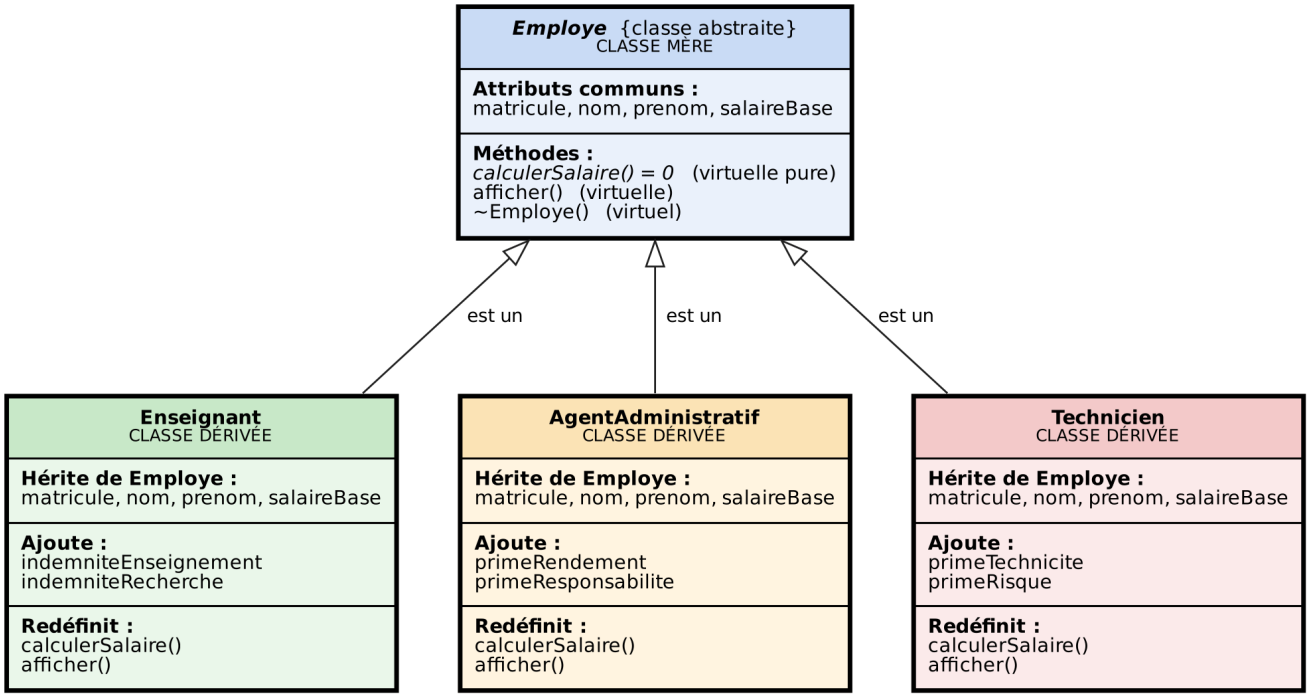


Figure 2 – Hiérarchie d'héritage : ce qui est hérité, ajouté et redéfini

Classe dérivée	Relation	Classe mère	Notation C++
Enseignant	est un	Employe	`class Enseignant : public Employe`
AgentAdministratif	est un	Employe	`class AgentAdministratif : public Employe`
Technicien	est un	Employe	`class Technicien : public Employe`

L'héritage est **public** : l'interface publique d'*Employe* reste accessible, ce qui rend possible le polymorphisme. Les attributs communs sont **protected** pour que les classes dérivées y accèdent directement.

3) Diagramme d'objets : la collection d'employés

Ce diagramme montre l'état du programme à l'exécution : le `vector<Employe*>` `personnel` contient quatre pointeurs de type *Employe**, mais les objets pointés appartiennent à des **classes différentes**.

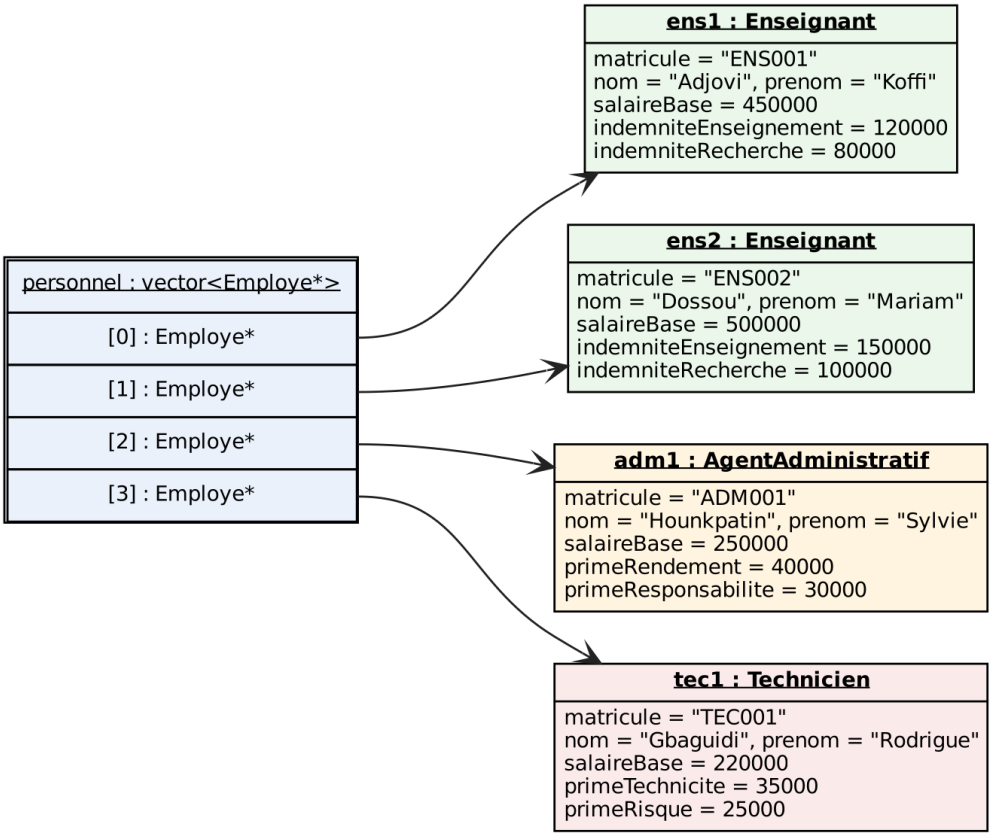


Figure 3 – Diagramme d'objets de la collection polymorphe

Un pointeur sur la classe mère peut désigner un objet de n'importe quelle classe dérivée : c'est ce qui permet de ranger plusieurs catégories d'employés dans une même collection.

4) Diagramme de séquence : liaison dynamique

La boucle `for (Employe* e : personnel) e->afficher();` appelle toujours la même méthode, mais c'est la version de la classe **réelle** de l'objet qui s'exécute.

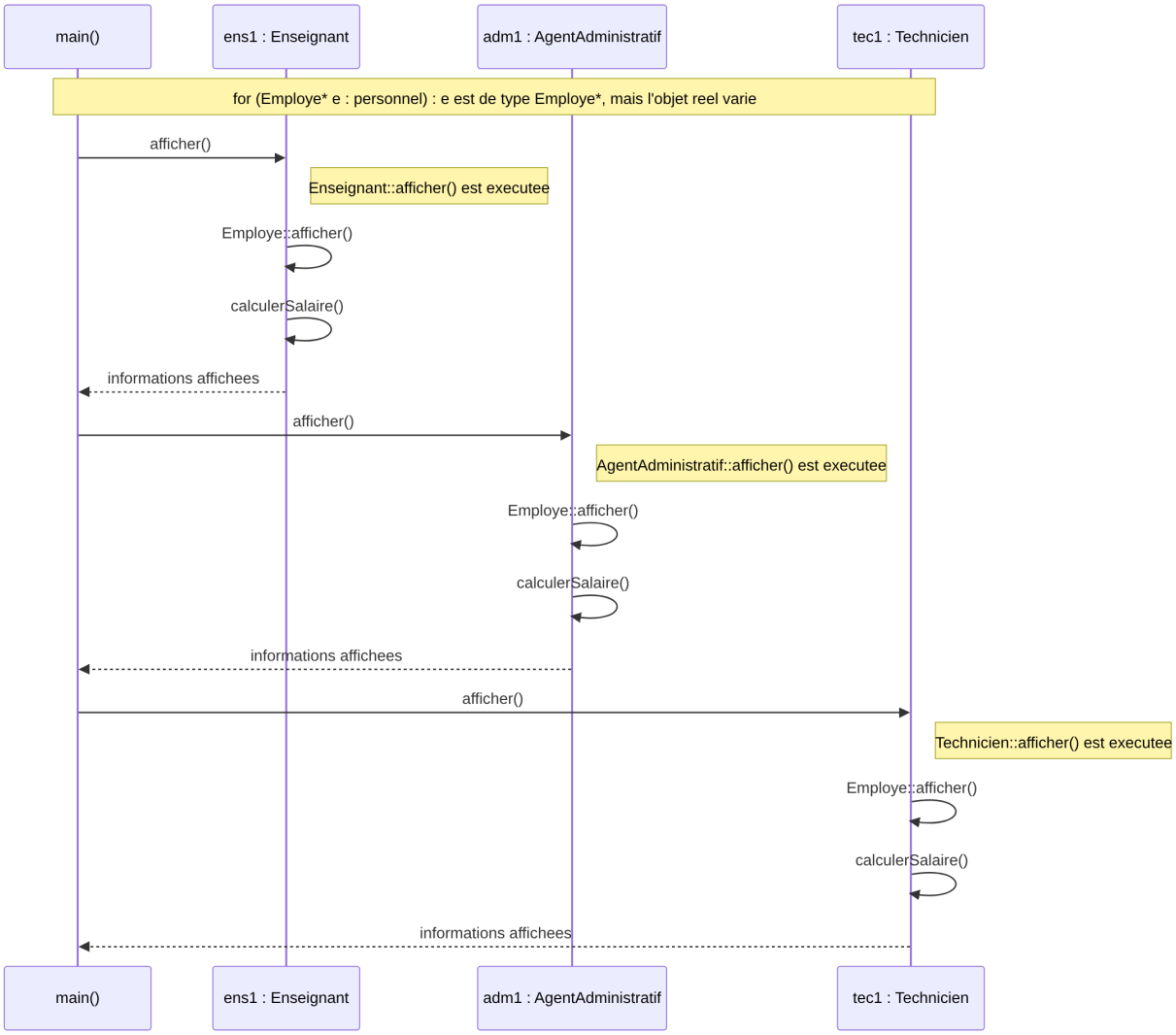


Figure 4 – Séquence d'appel polymorphe de afficher()

Lecture : pour chaque objet, la méthode `afficher()` de sa propre classe est exécutée ; elle appelle d'abord `Employe::afficher()` (informations communes) puis `calculerSalaire()` dans sa version redéfinie. Le choix se fait à l'exécution : c'est la **liaison dynamique**.